FLIGHT BOOKING SYSTEM

FBS

Final Project Report Presentation
Data Process and Software Modelling

Christian Cantos | 23188023

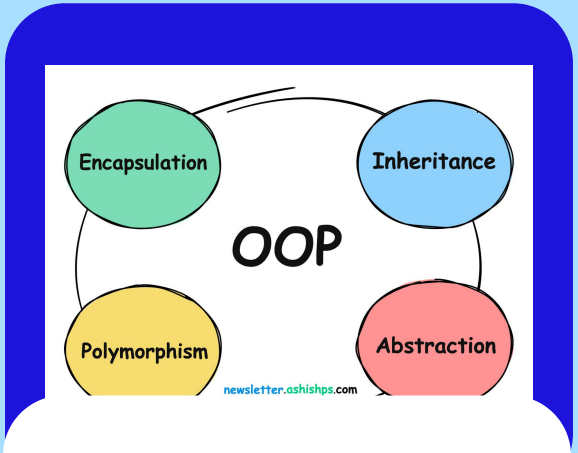
SUMMARY

- Project Overview
- System Architecture
- Refined UML Class Diagram
- Key Improvements
- Design Patterns
- Sequence Diagram
- Activity Diagram
- Demonstration
- Challenges & Lessons Learned
- Future Improvements

PROJECT OVERVIEW

C#

Developed in C#



OOP

Applied Objected Oriented Principles

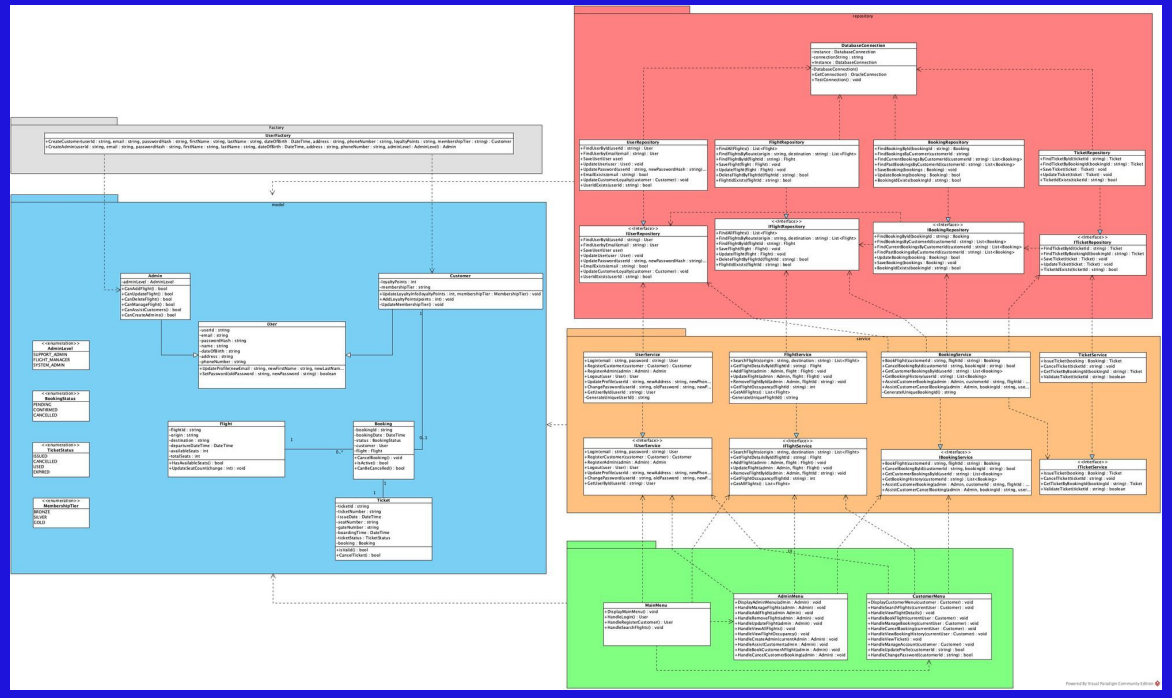
DP

Implementation of System Design

SYSTEM ARCHITECTURE

- Layered Architecture
- Learned from previous project
- Maintainability
- Separation of Concerns
- Reduces coupling

REFINED UML CLASS DIAGRAM



- Introduced Inheritance
- Specialised Subclasses
  - Customer
  - Admin
- Eliminated extensive role-checking
- Improved object oriented structure through abstraction and polymorphism.
- Role Checking
- New Ticket Entity
- Introduced Interfaces
- Improve separation of concerns
- Increase system maintainability, scalability, and extensibility
- Better reflects real-world airling booking operations

KEY IMPROVEMENTS

Initial Design

- Single User class with Role Enumeration
- Extensive role-checking
- No Ticket Entity
- Basic domain model
- Direct object creation using constructors
- No centralised connection management
- Less specialised responsibilities
- Limited extensibility for new user types
- Lower cohesion in user management
- Higher coupling between components

Refined Design

- Abstract user superclass with Customer and Admin subclasses
- Uses inheritance and polymorphism
- Added Ticket entity for ticket management
- More complete airline reservation model
- Factory Method Pattern through UserFactory
- Singleton Pattern through DatabaseConnection
- Clear separation of responsibilities
- Easier to add new user types and features
- Improved cohesion through specialised subclasses
- Reduced coupling through interfaces and layered architecture

DESIGN PATTERNS

- Singleton Pattern
- Factory Method

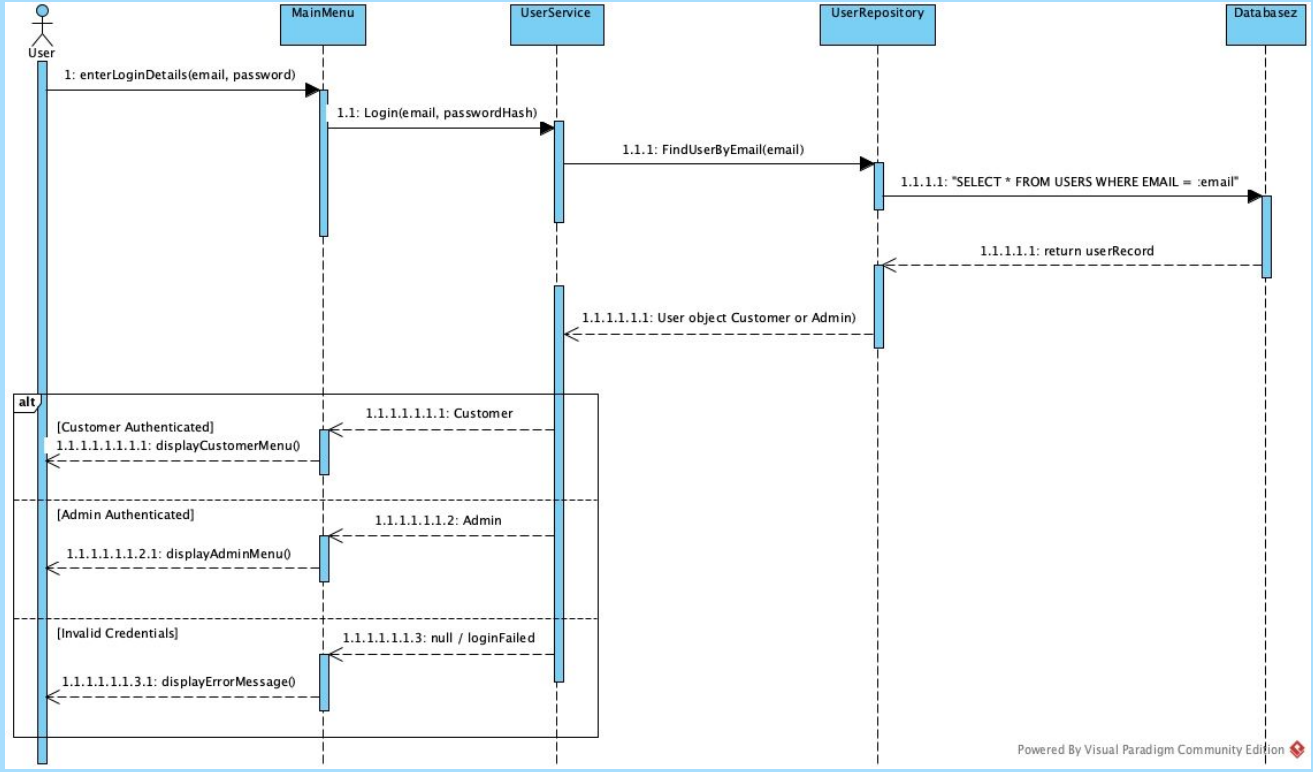
C# Implementation

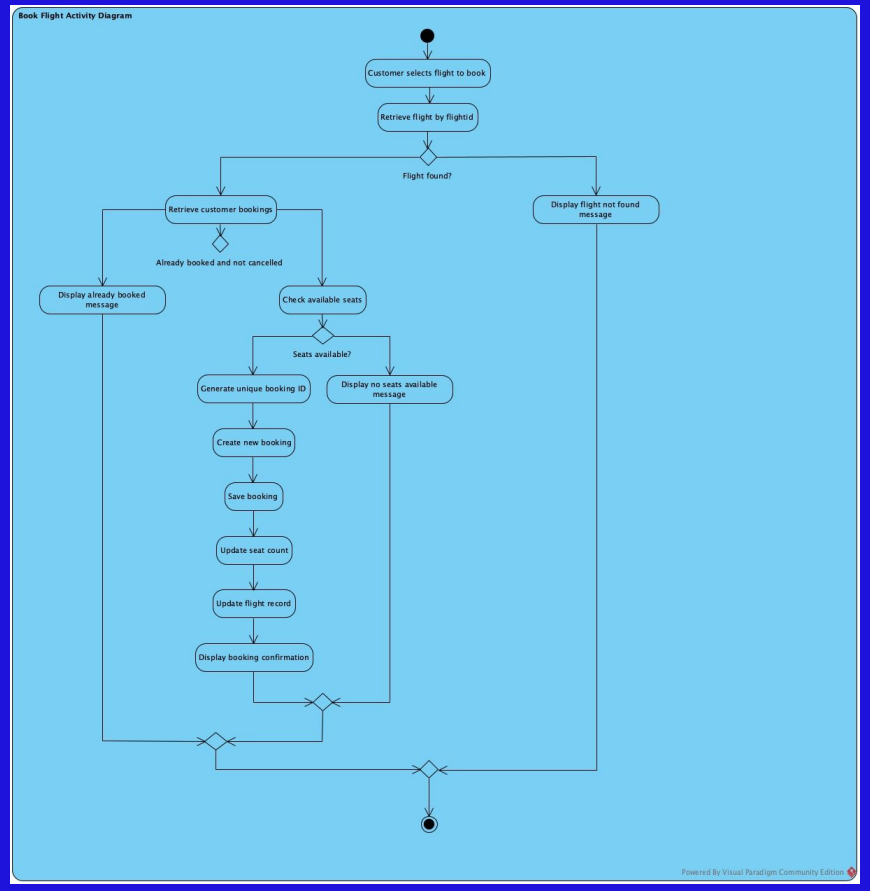
```
.cs > { } FlightBookingSystem.Factory > UserFactory > CreateAdmin(string, string, string, string, string, Date
1   using FlightBookingSystem.Model;
2
3   namespace FlightBookingSystem.Factory
4   {
        0 references
5       public static class UserFactory
6       {
            0 references
7           public static Customer CreateCustomer(
8               string userId,
9               string email,
10              string passwordHash,
11              string firstName,
12              string lastName,
13              DateTime dateOfBirth,
14              string address,
15              string phoneNumber,
16              int loyaltyPoints,
17              string membershipTier)
18          {
19              return new Customer(userId, email, passwordHash, firstName, lastName,
20                  dateOfBirth, address, phoneNumber);
21          }
22
            0 references
23          public static Admin CreateAdmin(
24              string userId,
25              string email,
26              string passwordHash,
27              string firstName,
28              string lastName,
29              DateTime dateOfBirth,
30              string address,
31              string phoneNumber,
32              AdminLevel adminLevel)
33          {
34              return new Admin(userId, email, passwordHash, firstName, lastName,
35                  dateOfBirth, address, phoneNumber, adminLevel);
36          }
37      }
38  }
```

```
Repository > DatabaseConnection.cs > { } FlightBookingSystem.Repository > DatabaseConnection > GetConnec
1   using Oracle.ManagedDataAccess.Client;
2
3   namespace FlightBookingSystem.Repository
4   {
        31 references
5       public sealed class DatabaseConnection
6       {
            3 references
7           private static DatabaseConnection? instance; // Singleton Pattern
            2 references
8           private readonly string connectionString;
9
10          // Private constructor prevents external instantiation
            1 reference
11          private DatabaseConnection()
12          {
13              connectionString = "User Id=system;Password=1234;Data Source=localhost:1521/xe;";
14          }
15
16          // Global access point
            27 references
17          public static DatabaseConnection Instance
18          {
19              get
20              {
21                  if(instance == null)
22                  {
23                      instance = new DatabaseConnection();
24                  }
25                  return instance;
26              }
27          }
28
            28 references
29          public OracleConnection GetConnection()
30          {
31              return new OracleConnection(connectionString);
32          }
33
            0 references
34          public void TestConnection()
35          {
36              using OracleConnection conn = GetConnection();
37              conn.Open();
38
39              Console.WriteLine("Database connected successfully!");
40              Console.WriteLine($"Data Source: {conn.DataSource}");
41
42              using OracleCommand cmd = new OracleCommand("SELECT USER FROM dual", conn);
43              string currentUser = cmd.ExecuteScalar()!.ToString()!;
44
45              Console.WriteLine($"Connected as: {currentUser}");
46          }
47      }
48  }
```

SEQUENCE DIAGRAM

Login Sequence Diagram





Book Flight Activity Diagram

Demonstration

1 - 2 mins

CHALLENGES & LESSON LEARNED

- Role Enum to Enheritance
- Oracle Database Integration
- Keeping UML aligned with code
- Importance of System Design

EFFECTIVE COMMUNICATION IS:

- PAYMENT PROCESSING
- ADDITIONAL USER TYPES
- SEAT SELECTION
- BETTER INTERFACE

THANK YOU!